**Santiago Castaño: 202524173**
**Tomás Rivera: 202520971**
**Sofía Muñoz: 202521447**
**https://github.com/scasta06/Proyecto-1.git**

**Entrega 2: Diseño e Implementación**

**Organización**

El sistema está organizado por áreas para repartir las responsabilidades y hacer más fácil el manejo de la información. El diagrama tiene las clases que representan a los usuarios, los deportes, las instalaciones, las competencias, las actividades, las ventas y el inventario.

La lógica del diagrama está organizada en clases que se llaman gestores. Cada gestor se encarga de un área y antes de hacer cambios, revisa que se cumplan las reglas. Por ejemplo, el GestorInstalaciones es el encargado de las reservas, en cambio, GestorCompetencia se encarga de las competencias. Esto ayuda a que una clase no se encargue de todo y sea más organizado; ClubDeportivo conecta los distintos gestores.

En las consolas, los usuarios ingresan los datos, eligen las acciones que quieren hacer y ven los resultados. La persistencia es la encargada de guardar y recuperar los datos.

**Diagramas de clases y decisiones de diseño**

El diagrama de clases de diseño muestra cómo está organizado el sistema, las relaciones entre las clases, los atributos y los métodos, mientras que el diagrama de clases de alto nivel solo muestra la organización y las relaciones entre clases. Estos tienen diferentes própositos, el diagrama de diseño permite ver detalladamente la información de cada clase y sus funciones, y el de alto nivel ayuda a entender el sistema de forma general.

En las clases que comparten atributos o métodos, utilizamos herencia. Por ejemplo, Administrador, Fisioterapeuta y Entrenador heredan de Empleado, y así no hay necesidad de repetir información y ya en cada clase se agrega, si es necesario, lo específico de cada uno.

Para las sesiones de entrenamiento, los partidos y las reservas, creamos una clase que se llama Evento Decidimos hacer esto ya que estas tienen en común fecha, horario y duración, así no repetimos información y es más fácil mirar si dos actividades se cruzan en una misma instalación.

También, separamos las estadísticas por deporte ya que son diferentes y no todos los deportes registran lo mismo. Por ejemplo, en fútbol se guardan los goles y las asistencias, mientras que en voleibol se guardan los aces y los bloqueos. Cada tipo de estadística calculo los puntos para el ranking.

Creamos tres clases para las inscripciones, creamos InscripcionEquipo, InscripcionCompetencia e InscripcionAcademia, en estas clases se guarda infromación de la relación entre el socio con la actividad, como la fecha, los puntos acumulados, posición, número de camiseta, etc.

En la parte de comercio, creamos una clase que se llama PuntoVenta y de esta clase heredan Tienda y Cafetería ya que tienen métodos y atributos en común pero cada una tiene sus reglas de impuestos, descuentos y propina. También separamos Producto y Existencia, Producto guarda el código, el nombre y el precio, y Existencia guarda la cantidad y el StockMinimo y así, separados sabemos que productos hay y cuantas unidades quedan en lugares diferentes.

**Gestores**

- GestorUsuarios permite iniciar sesión, registrar y buscar usuarios y manejar las sanciones.

- GestorInstalaciones busca qué instalaciones están disponibles y puede crear y cancelar reservas.

- GestorDeportivo inscribe equipos y academias, programa los entrenamientos y los partidos de práctica, registra la asistencia y las estadísticas y consulta el ranking.

- GestorPersonal asigna turnos, aprueba o rechaza los cambios de turno, y revisa las solicitudes de cambio de turno.

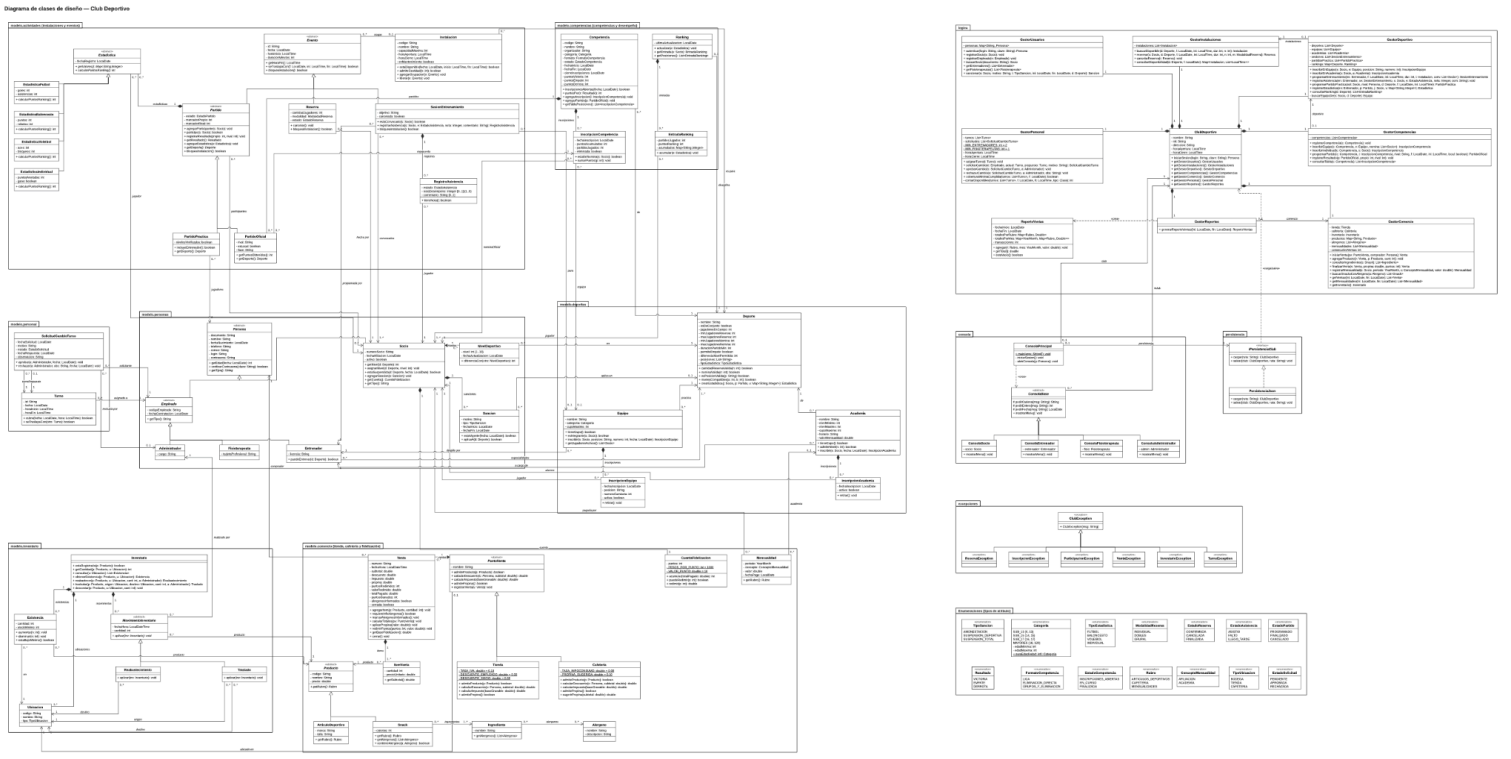
- GestorCompetencias registra las competencias, se encarga de inscribir a los jugadores y equipos, programa los partidos, registra los resultados y consulta las tablas de posiciones.

- GestorReportes se encarga de generar los reportes de las ventas.

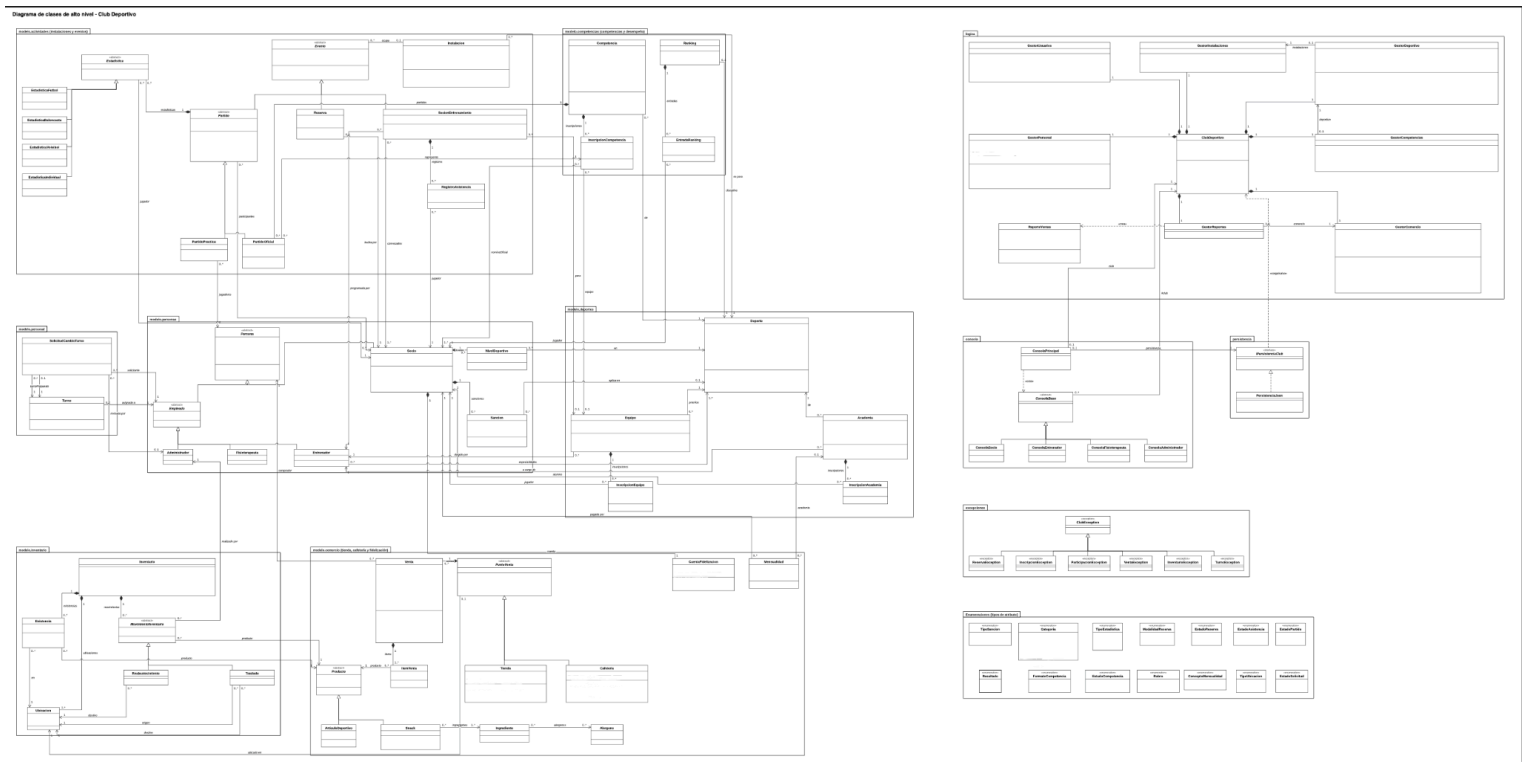
- GestorComercio se encarga de las ventas, de agregar productos, de registrar la mensualidad y de buscar los snacks con alérgenos.

Las clases del modelo revisan esto por aparte y los gestores utilizan estos métodos para completar el proceso. Por ejemplo, Instalacion revisa si está disponible y Equipo revisa si tiene cupo.

## Diagrama de clases de diseño




## Diagrama de clases de alto nivel



**Diagramas de secuencia**

Para los diagramas de secuencia escogimos 11 funcionalidades ya que en ellas se usan varias clases y se necesita revisar que cumplan ciertas reglas.

- **SD-01 Socio reserva una instalación:** Revisa si el socio tiene una sanción, la cantidad de reserva (la cantidad de jugadores), y si hay una instalación disponible. Si todo se cumple, se hace la reserva.

- **SD-02 Socio programa un partido de práctica:** Revisa si el socio está suspendido, mira los niveles de los socios y si son compatibles, busca si una instalación está disponible. Al mirar los niveles y la compatibilidad hay una excepción de nivel cuando el rival es un entrenador. Si todo está bien y las reglas se cumplen, se programa el partido.

- **SD-03 Entrenador programa una sesión de entrenamiento:** Se revisa la fecha, hora, duración, la disponibilidad de la instalación, los jugadores convocados, la especialidad del entrenador y si algún jugador tiene una sanción. Si todo se cumple y está bien, se programa la sesión de entrenamiento.

- **SD-04 Entrenador registra asistencia y desempeño:** Se elige la sesión, el estado, el jugador y la nota, se registra asistencia, se verifica que el entrenador sea el que programó la sesión, que el jugador esté convocado y se revisa que la nota esté entre 1 a 5. Si todo se cumple, se registra la asistencia.

- **SD-05 Administrador aprueba un cambio de turno:** Se revisa el estado de la solicitud y se simula el cambio antes de aceptarlo para revisar que se cumpla la cobertura minima de 2 entrenadores y 1 fisioterapeuta en toda la franja del horario de atención. Si todo se cumple, el administrador decide si acepta o rechaza la solicitud de cambio de turno. Si la solicitud es aprobada se actualizan los turnos.

- **SD-06 Adinstrador consulta el reporte de ventas por rubro y periodo:** Se genera un reporte de ventas, reúne las ventas y las mensualidades del periodo solicitado y organiza la información por rubro y por mes.

- **SD-07 Socio se inscribe a un equipo:** Se revisa si el socio está suspendido, la categoría por edad, si el socio no pertenece a otro equipo del mismo deporte, el cupo, la posición y el número de la camiseta. Si todo se cumple la inscripción queda confirmada.

- **SD-08 Entrenador registra estadísticas de un partido y se actualiza el ranking:** Se revisa el estado del partido (si ya finalizó), si el jugador participó, el deporte y después se crea la estadística correspondiente y se actualiza el ranking.

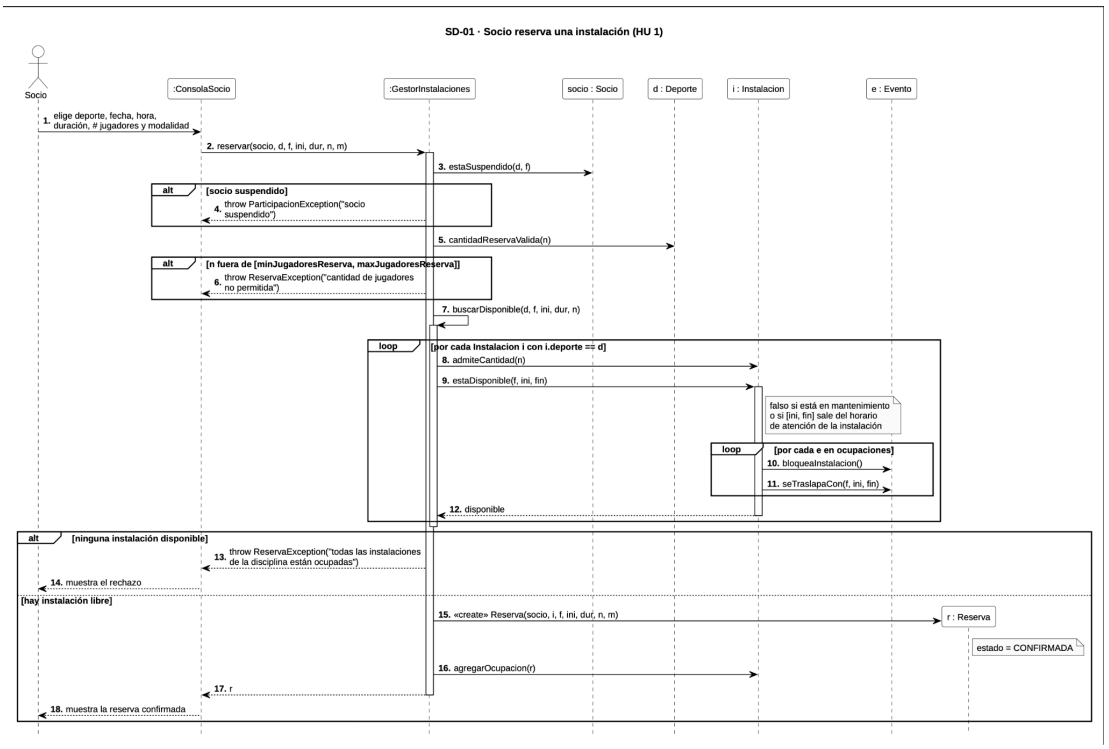
- **SD-09 Socio compra un snack en la cafetería:** Se inicia y se crea la venta, verifica que el producto tenga existencia, se revisan los ingredientes y alérgenos, se calcula

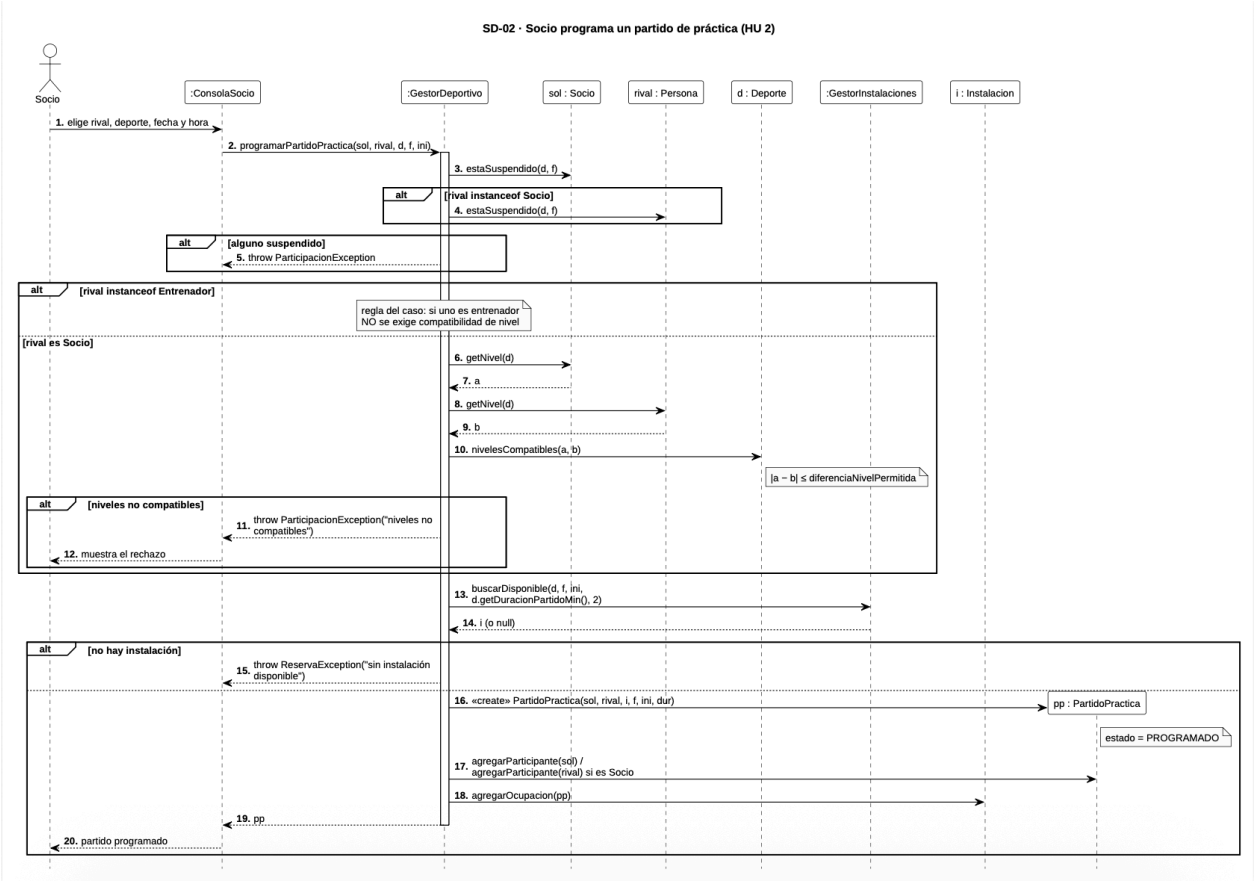
el impuesto y la propina, se revisa si se pueden redimir puntos, se acumulan puntos y si todo está bien se registra la venta y se entrega una factura con el desglose.

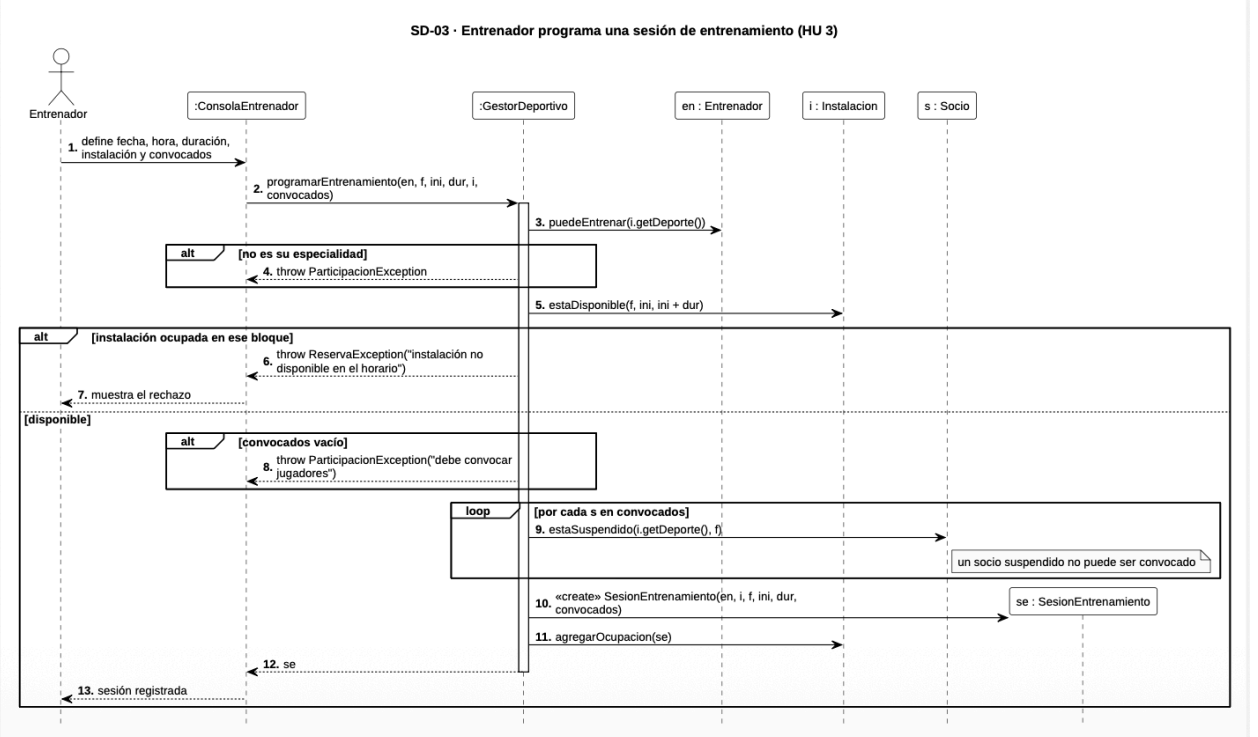
- **SD-10 Administrador reabastece y traslada inventario entre ubicaciones:** Revisa la ubicación, el producto y la cantidad,  actualiza las existencias y se registra el movimiento realizado.

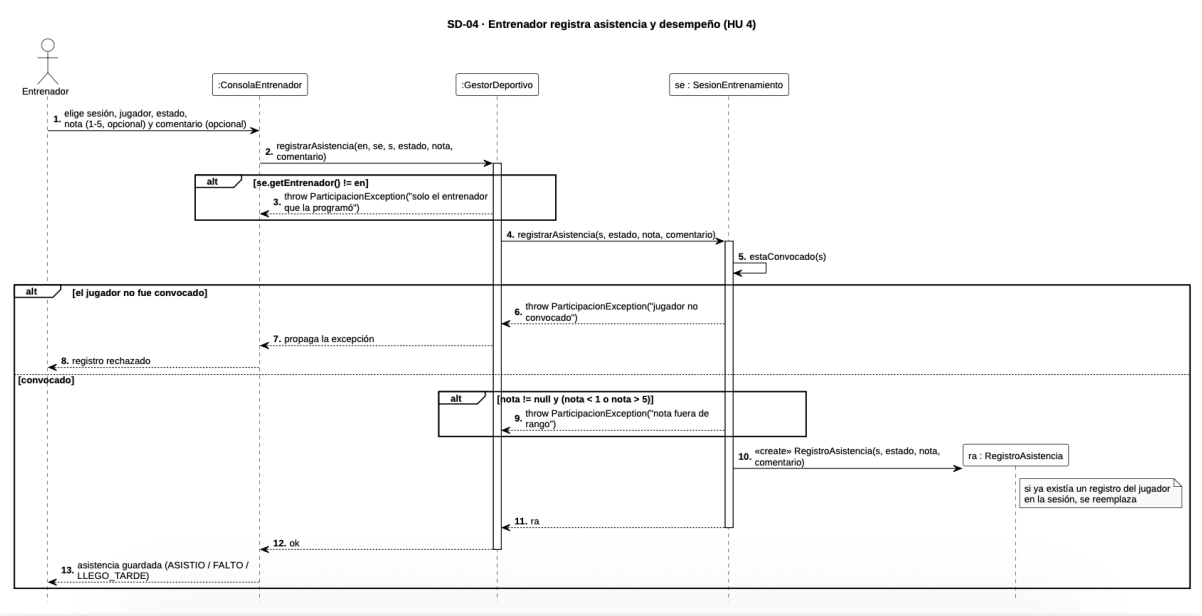
- **SD-11 Entrenador inscribe un equipo en una competencia externa con su nómina oficial:** Se elige la competencia, el equipo y los jugadores de la nómina, se revisa si las inscripciones están abiertas, el deporte, si la nómina es válida, los integrantes, si hay un jugador suspendido y si la edad corresponde con la categoría de la competencia. Si todo se cumple el equipo queda inscrito en la competencia con la nómina oficial.

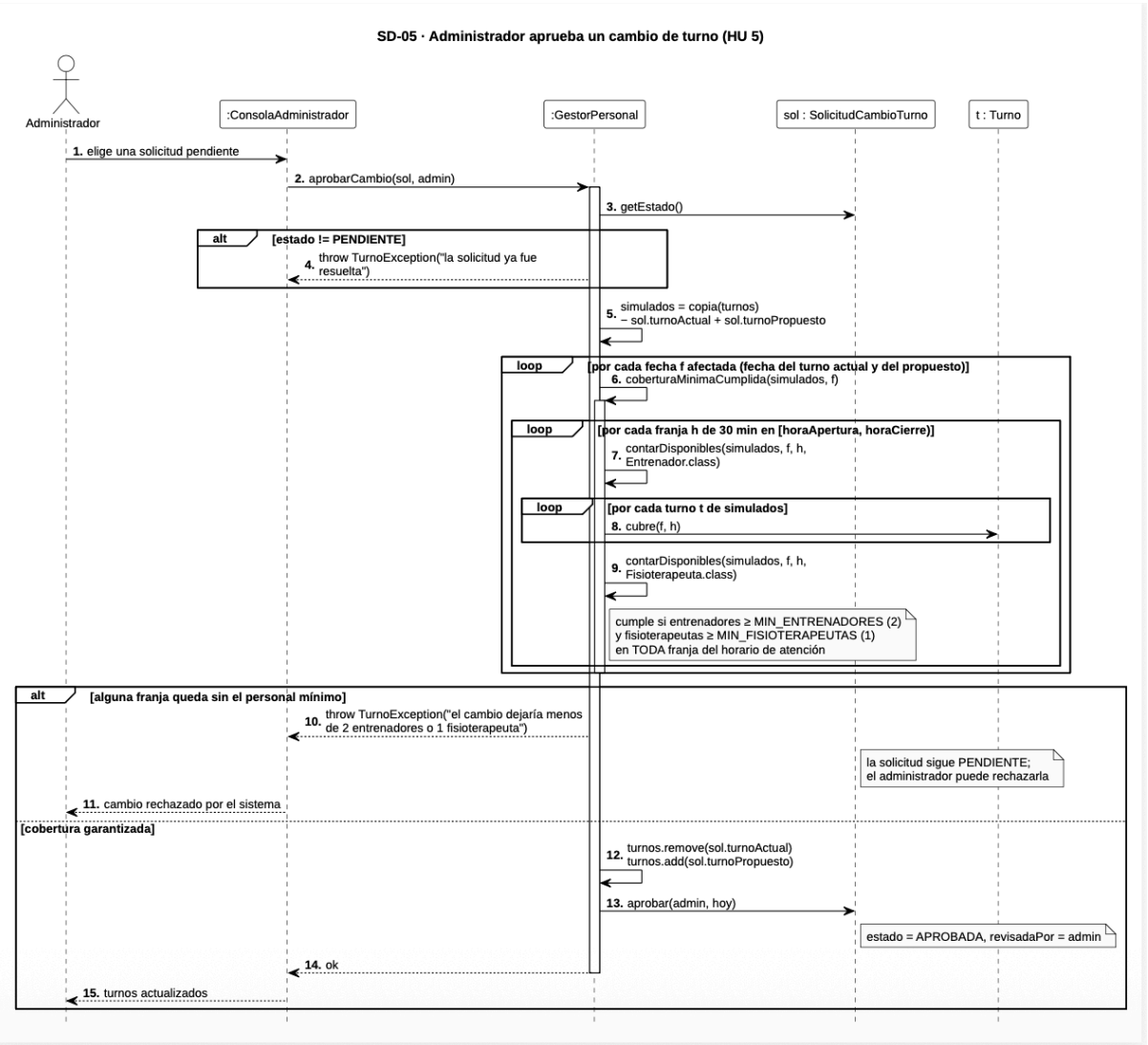
Los diagramas muestran los pasos de una operación válida y lo que pasa cuando una operación debe rechazarse porque incumple alguna regla. Escogimos estas funcionalidades porque un error en alguno de los procesos podría causar diferentes inconvenientes, como reservas a la misma hora en una misma instalación, inscripciones que no cumplen las reglas o turnos sin suficiente personal.



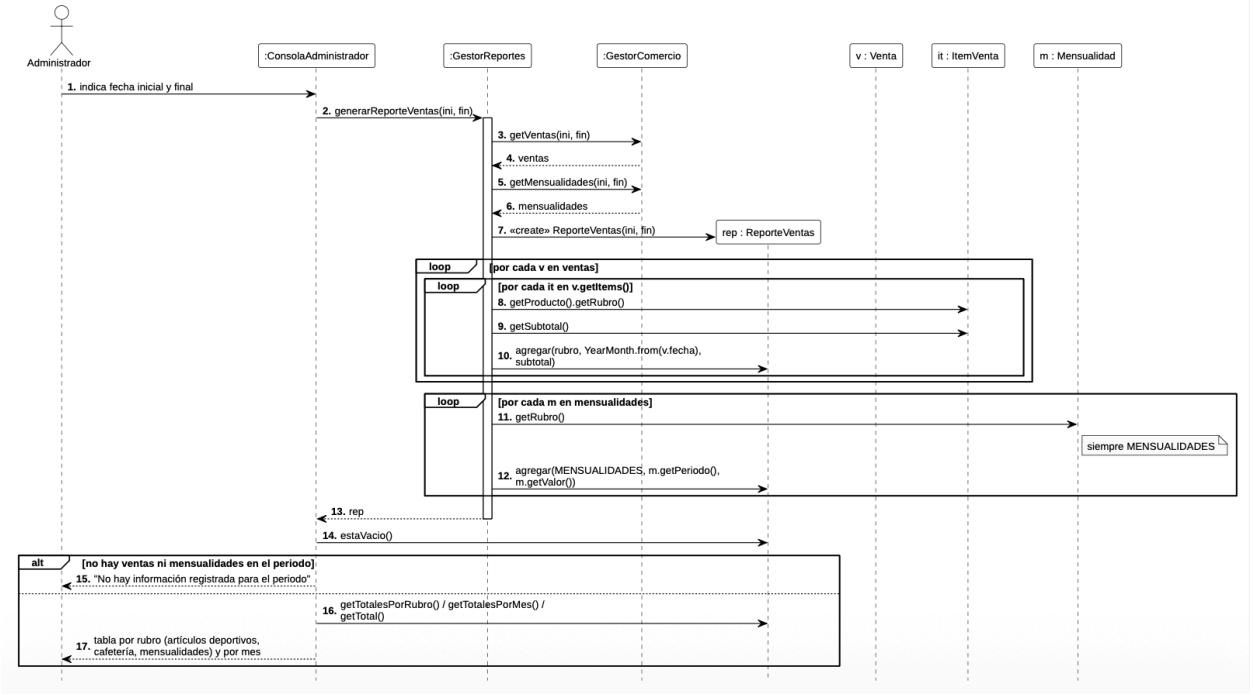

SD-02 · Socio programa un partido de práctica (HU 2)
Socio
:ConsolaSocio
:GestorDeportivo
sol : Socio
rival : Persona
d : Deporte
:GestorInstalaciones
i : Instalacion
1. elige rival, deporte, fecha y hora
2. programarPartidoPractica(sol, rival, d, f, ini)
3. estaSuspendido(d, f)
alt
[rival instanceof Socio]
4. estaSuspendido(d, f)
alt
[alguno suspendido]
5. throw ParticipacionException
alt
[rival instanceof Entrenador]
regla del caso: si uno es entrenador
NO se exige compatibilidad de nivel
[rival es Socio]
6. getNivel(d)
7. a
8. getNivel(d)
9. b
10. nivelesCompatibles(a, b)
|a − b| ≤ diferenciaNivelPermitida
alt
[niveles no compatibles]
11. throw ParticipacionException("niveles no compatibles")
12. muestra el rechazo
13. buscarDisponible(d, f, ini, d.getDuracionPartidoMin(), 2)
14. i (o null)
alt
[no hay instalación]
15. throw ReservaException("sin instalación disponible")
16. «create» PartidoPractica(sol, rival, i, f, ini, dur)
pp : PartidoPractica
estado = PROGRAMADO
17. agregarParticipante(sol) / agregarParticipante(rival) si es Socio
18. agregarOcupacion(pp)
19. pp
20. partido programado


SD-03 · Entrenador programa una sesión de entrenamiento (HU 3)
Entrenador
:ConsolaEntrenador
:GestorDeportivo
en : Entrenador
i : Instalacion
s : Socio
1. define fecha, hora, duración, instalación y convocados
2. programarEntrenamiento(en, f, ini, dur, i, convocados)
3. puedeEntrenar(i.getDeporte())
alt
[no es su especialidad]
4. throw ParticipacionException
5. estaDisponible(f, ini, ini + dur)
alt
[instalación ocupada en ese bloque]
6. throw ReservaException("instalación no disponible en el horario")
7. muestra el rechazo
[disponible]
alt
[convocados vacío]
8. throw ParticipacionException("debe convocar jugadores")
loop
[por cada s en convocados]
9. estaSuspendido(i.getDeporte(), f)
un socio suspendido no puede ser convocado
10. «create» SesionEntrenamiento(en, i, f, ini, dur, convocados)
se : SesionEntrenamiento
11. agregarOcupacion(se)
12. se
13. sesión registrada

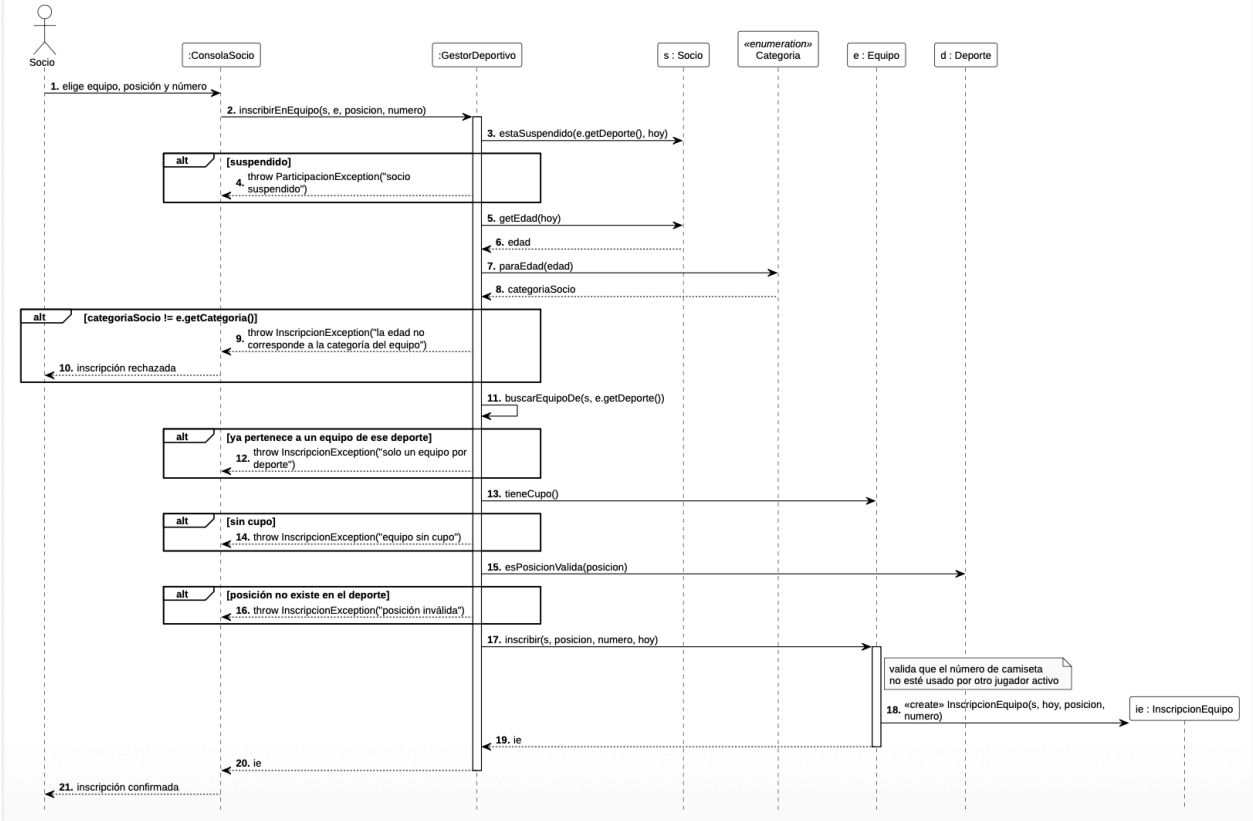

SD-04 · Entrenador registra asistencia y desempeño (HU 4)
Entrenador
:ConsolaEntrenador
:GestorDeportivo
se : SesionEntrenamiento
1. elige sesión, jugador, estado, nota (1-5, opcional) y comentario (opcional)
2. registrarAsistencia(en, se, s, estado, nota, comentario)
alt
[se.getEntrenador() != en]
3. throw ParticipacionException("solo el entrenador que la programó")
4. registrarAsistencia(s, estado, nota, comentario)
5. estaConvocado(s)
alt
[el jugador no fue convocado]
6. throw ParticipacionException("jugador no convocado")
7. propaga la excepción
8. registro rechazado
[convocado]
alt
[nota != null y (nota < 1 o nota > 5)]
9. throw ParticipacionException("nota fuera de rango")
10. «create» RegistroAsistencia(s, estado, nota, comentario)
ra : RegistroAsistencia
si ya existía un registro del jugador en la sesión, se reemplaza
11. ra
12. ok
13. asistencia guardada (ASISTIO / FALTO / LLEGO_TARDE)


SD-05 · Administrador aprueba un cambio de turno (HU 5)
Administrador
:ConsolaAdministrador
:GestorPersonal
sol : SolicitudCambioTurno
t : Turno
1. elige una solicitud pendiente
2. aprobarCambio(sol, admin)
3. getEstado()
alt
[estado != PENDIENTE]
4. throw TurnoException("la solicitud ya fue resuelta")
5. simulados = copia(turnos) − sol.turnoActual + sol.turnoPropuesto
loop
[por cada fecha f afectada (fecha del turno actual y del propuesto)]
6. coberturaMinimaCumplida(simulados, f)
loop
[por cada franja h de 30 min en [horaApertura, horaCierre)]
7. contarDisponibles(simulados, f, h, Entrenador.class)
loop
[por cada turno t de simulados]
8. cubre(f, h)
9. contarDisponibles(simulados, f, h, Fisioterapeuta.class)
cumple si entrenadores ≥ MIN_ENTRENADORES (2) y fisioterapeutas ≥ MIN_FISIOTERAPEUTAS (1) en TODA franja del horario de atención
alt
[alguna franja queda sin el personal mínimo]
10. throw TurnoException("el cambio dejaría menos de 2 entrenadores o 1 fisioterapeuta")
la solicitud sigue PENDIENTE; el administrador puede rechazarla
11. cambio rechazado por el sistema
[cobertura garantizada]
12. turnos.remove(sol.turnoActual) turnos.add(sol.turnoPropuesto)
13. aprobar(admin, hoy)
estado = APROBADA, revisadaPor = admin
14. ok
15. turnos actualizados

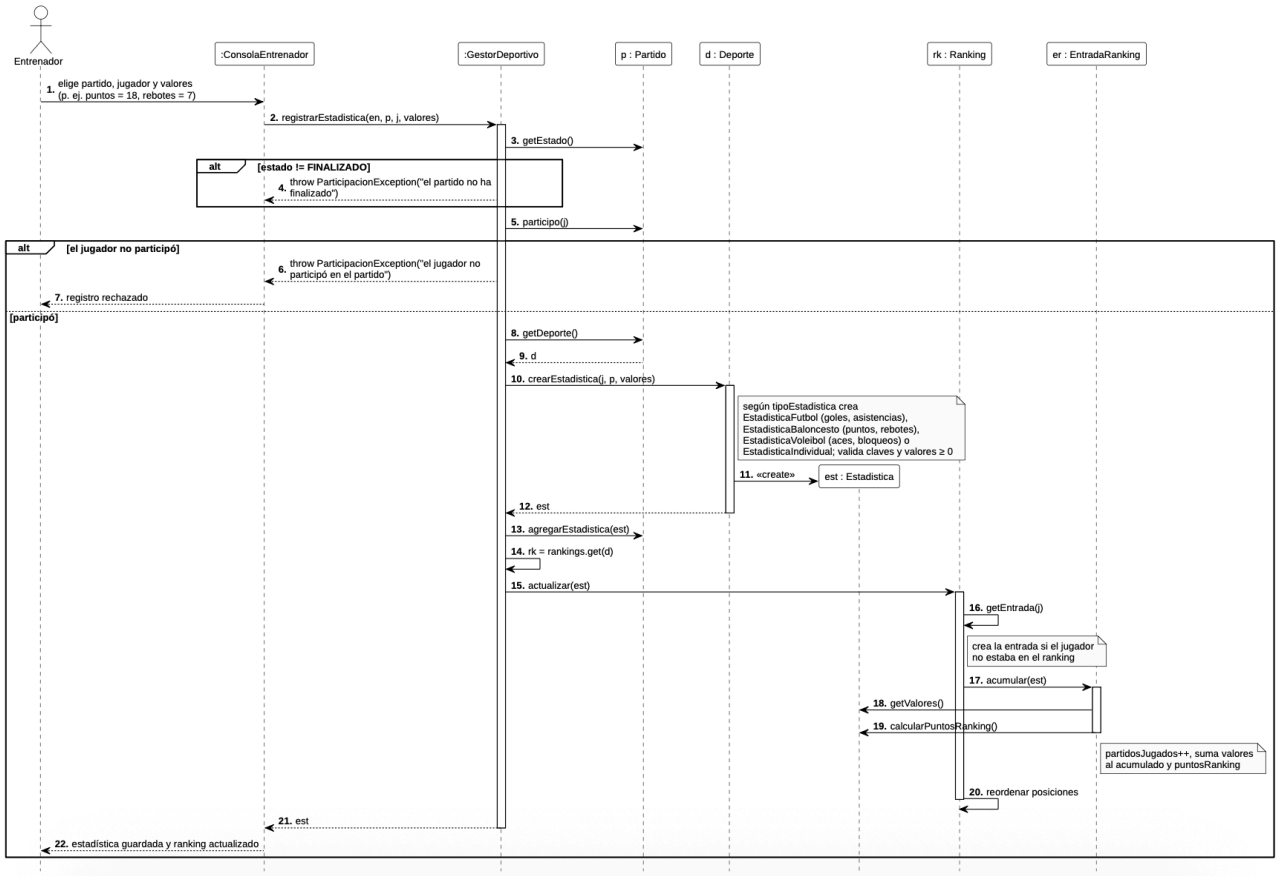
**SD-06 · Administrador consulta el reporte de ventas por rubro y periodo (HU 6)**

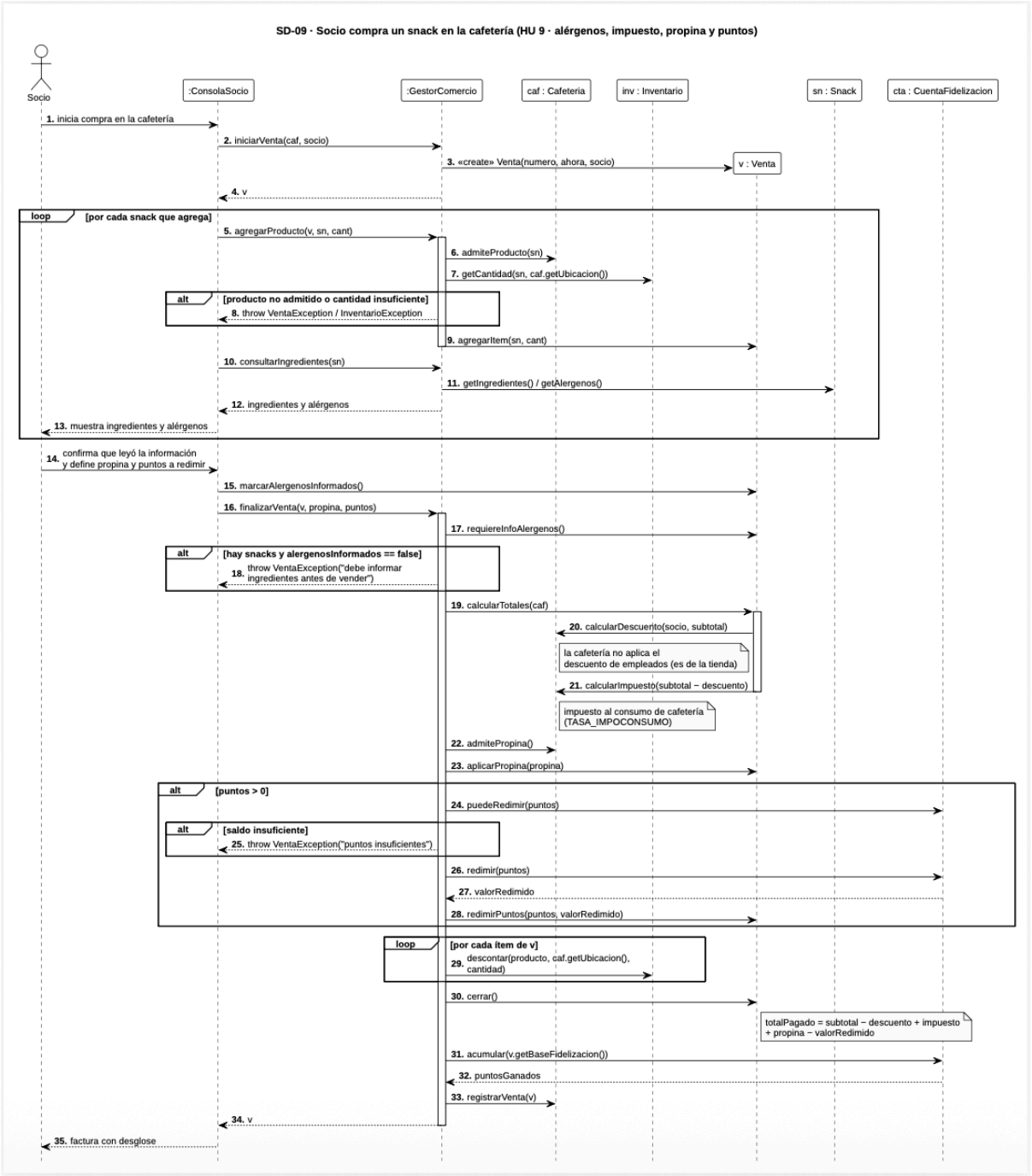


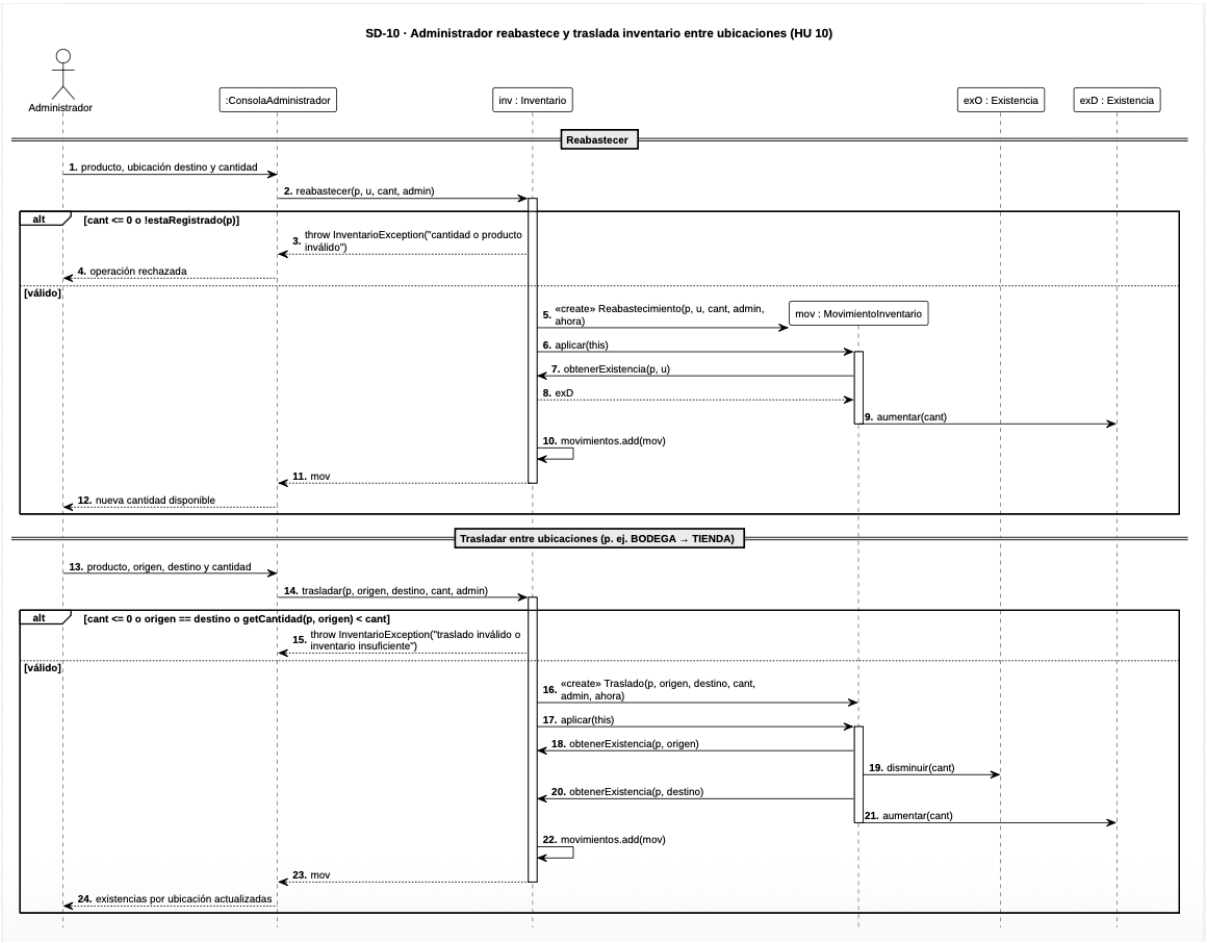

**SD-07 · Socio se inscribe en un equipo (HU 7)**

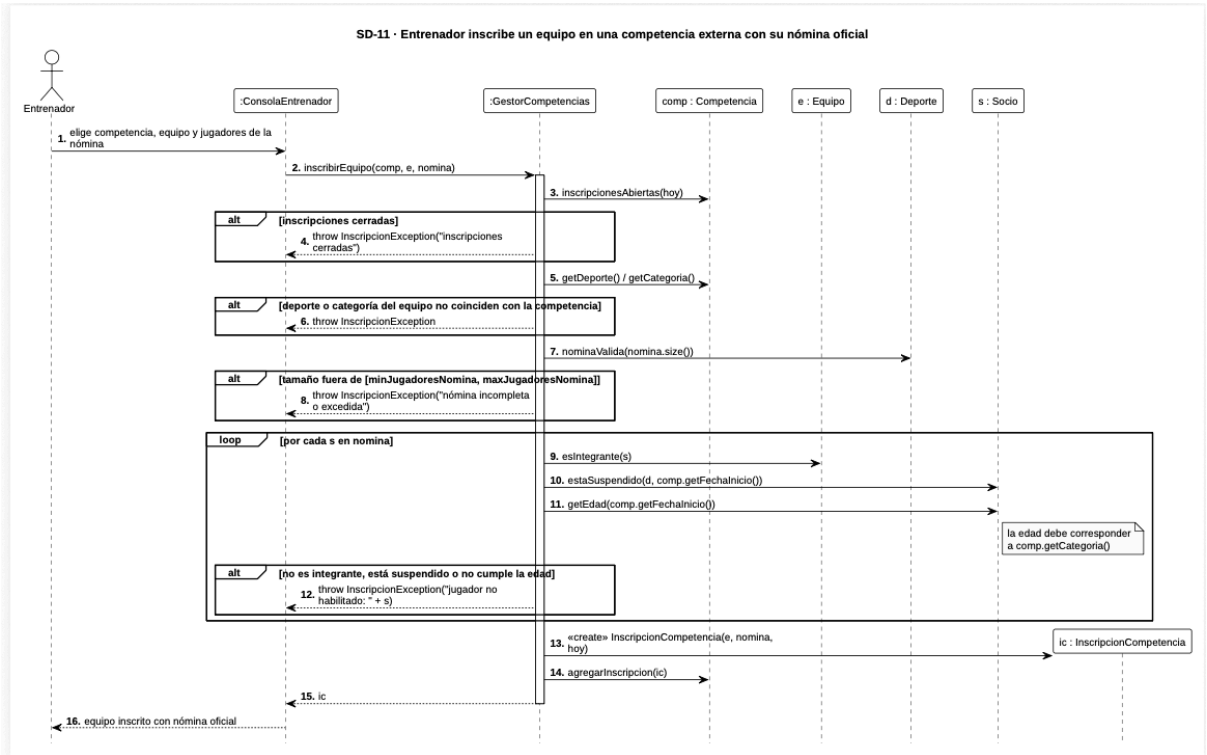


**SD-08 · Entrenador registra estadísticas de un partido y se actualiza el ranking (HU 8)**




SD-09 · Socio compra un snack en la cafetería (HU 9 · alérgenos, impuesto, propina y puntos)
Socio
:ConsolaSocio
:GestorComercio
caf : Cafeteria
inv : Inventario
v : Venta
sn : Snack
cta : CuentaFidelizacion
1. inicia compra en la cafetería
2. iniciarVenta(caf, socio)
3. «create» Venta(numero, ahora, socio)
4. v
loop
[por cada snack que agrega]
5. agregarProducto(v, sn, cant)
6. admiteProducto(sn)
7. getCantidad(sn, caf.getUbicacion())
alt
[producto no admitido o cantidad insuficiente]
8. throw VentaException / InventarioException
9. agregarItem(sn, cant)
10. consultarIngredientes(sn)
11. getIngredientes() / getAlergenos()
12. ingredientes y alérgenos
13. muestra ingredientes y alérgenos
14. confirma que leyó la información y define propina y puntos a redimir
15. marcarAlergenosInformados()
16. finalizarVenta(v, propina, puntos)
17. requiereInfoAlergenos()
alt
[hay snacks y alergenosInformados == false]
18. throw VentaException("debe informar ingredientes antes de vender")
19. calcularTotales(caf)
20. calcularDescuento(socio, subtotal)
la cafetería no aplica el descuento de empleados (es de la tienda)
21. calcularImpuesto(subtotal − descuento)
impuesto al consumo de cafetería (TASA_IMPOCONSUMO)
22. admitePropina()
23. aplicarPropina(propina)
alt
[puntos > 0]
24. puedeRedimir(puntos)
alt
[saldo insuficiente]
25. throw VentaException("puntos insuficientes")
26. redimir(puntos)
27. valorRedimido
28. redimirPuntos(puntos, valorRedimido)
loop
[por cada ítem de v]
29. descontar(producto, caf.getUbicacion(), cantidad)
30. cerrar()
totalPagado = subtotal − descuento + impuesto + propina − valorRedimido
31. acumular(v.getBaseFidelizacion())
32. puntosGanados
33. registrarVenta(v)
34. v
35. factura con desglose


SD-10 · Administrador reabastece y traslada inventario entre ubicaciones (HU 10)
Administrador
:ConsolaAdministrador
inv : Inventario
exO : Existencia
exD : Existencia
Reabastecer
1. producto, ubicación destino y cantidad
2. reabastecer(p, u, cant, admin)
alt
[cant <= 0 o !estaRegistrado(p)]
3. throw InventarioException("cantidad o producto inválido")
4. operación rechazada
[válido]
5. «create» Reabastecimiento(p, u, cant, admin, ahora)
mov : MovimientoInventario
6. aplicar(this)
7. obtenerExistencia(p, u)
8. exD
9. aumentar(cant)
10. movimientos.add(mov)
11. mov
12. nueva cantidad disponible
Trasladar entre ubicaciones (p. ej. BODEGA → TIENDA)
13. producto, origen, destino y cantidad
14. trasladar(p, origen, destino, cant, admin)
alt
[cant <= 0 o origen == destino o getCantidad(p, origen) < cant]
15. throw InventarioException("traslado inválido o inventario insuficiente")
[válido]
16. «create» Traslado(p, origen, destino, cant, admin, ahora)
17. aplicar(this)
18. obtenerExistencia(p, origen)
19. disminuir(cant)
20. obtenerExistencia(p, destino)
21. aumentar(cant)
22. movimientos.add(mov)
23. mov
24. existencias por ubicación actualizadas


SD-11 · Entrenador inscribe un equipo en una competencia externa con su nómina oficial
Entrenador
:ConsolaEntrenador
:GestorCompetencias
comp : Competencia
e : Equipo
d : Deporte
s : Socio
1. elige competencia, equipo y jugadores de la nómina
2. inscribirEquipo(comp, e, nomina)
3. inscripcionesAbiertas(hoy)
alt
[inscripciones cerradas]
4. throw InscripcionException("inscripciones cerradas")
5. getDeporte() / getCategoria()
alt
[deporte o categoría del equipo no coinciden con la competencia]
6. throw InscripcionException
7. nominaValida(nomina.size())
alt
[tamaño fuera de [minJugadoresNomina, maxJugadoresNomina]]
8. throw InscripcionException("nómina incompleta o excedida")
loop
[por cada s en nomina]
9. esIntegrante(s)
10. estaSuspendido(d, comp.getFechaInicio())
11. getEdad(comp.getFechaInicio())
la edad debe corresponder a comp.getCategoria()
alt
[no es integrante, está suspendido o no cumple la edad]
12. throw InscripcionException("jugador no habilitado: " + s)
13. «create» InscripcionCompetencia(e, nomina, hoy)
ic : InscripcionCompetencia
14. agregarInscripcion(ic)
15. ic
16. equipo inscrito con nómina oficial

**Excepciones y enumeraciones**

Para identificar problemas que pueden aparecer en cada área utilizamos excepciones como ReservaException, InscripcionException, ParticipacionException, VentaException, InventarioException y TurnoException.

Estas excepciones nos ayudan a mostrarle al usuario porqie no se pudo realizar o completar una operación. Por ejemplo con las reservas, si no hay una instalación libre se rechaza la reserva. La consola le muestra el mensaje de porqué no se puedo realizar la operación al usuario.

También usamos enumeraciones para las opciones que ya están definidas, por ejemplo los tipos de sanciones, el tipo de estadística, el estado de la competencia, entre otras. Así solo toca escoger entre esas opciones y no hay que escribir lo mismo de manera diferente.

**Persistencia**

Utilizamos la interfaz IPersistenciaClub Para cargar y guardar los datos; esto tiene como métodos cargar y salvar. La clase PersistenciaJson realiza esto mismo solo que utilizando archivos JSON.

Cuando se carga el programa, este recupera la información y vuelve a crear los objetos con sus relaciones y al guardar el programa se conserva la información para que cuando se vuelva a ejecutar el programa esta siga ahí. Estos archivos van en una carpeta diferente a la del código.

Decidimos separar la persistencia para que así las clases de deportes y ventas no tengan que leer y escribir archivos. Además si queremos hacer un cambio en la forma en la que se guardan los datos no tenemos que cambiar toda la lógica del sistema.

*El diagrama de secuencia fue hecho por nosotros con la ayuda de Plant UML y Claude. Promp utilizado “Ayudame a corregir este diagrama de secuencia, se ve muy desordenado pero tengo las ideas claras :SD-01_reservar.png: el socio reserva una instalación (HU 1).*
*SD-02_partido_practica.png: el socio programa un partido de práctica (HU 2).*
*SD-03_entrenamiento.png: el entrenador programa un entrenamiento (HU 3).*
*SD-04_asistencia.png: el entrenador registra asistencia y nota (HU 4).*
*SD-05_cambio_turno.png: el administrador aprueba un cambio de turno (HU 5).*
*SD-06_reporte.png: reporte de ventas por rubro y periodo (HU 6).*
*SD-07_inscribir_equipo.png: el socio se inscribe en un equipo (HU 7).*
*SD-08_estadisticas.png: el entrenador registra estadísticas y se actualiza el ranking (HU 8).*
*SD-09_venta_cafeteria.png: compra en la cafetería con alérgenos, impuesto, propina y puntos (HU 9).*
*SD-10_inventario.png: reabastecer y trasladar inventario (HU 10).*
*SD-11_competencia.png: inscribir un equipo en una competencia externa.”*

*El diagrama de clases de diseño y el diagrama de clases de alto nivel fueron hechos por nosotros, se utilizó IA para organizar y corregir.*